

CPT204 Data Structures and Algorithms - 23-24F

Part A. Gap-fill Questions

1.

A __ variable stores the actual values of primitive data types directly, while a __ variable stores the address of an object in memory.

sol:

primitive reference

2.

__ is a feature in Java where a subclass provides a specific implementation of a method that is already provided by its parent class, whereas __ is a feature that allows a class to have more than one method having the same name, but with different parameter lists.

sol:

method overriding (Overriding) method overloading (Overloading)

3.

The outputs of the following code are __ and __.

```
public class Test {  
    public static void main(String[] args) {  
        Object o1 = new Object();  
        Object o2 = new Object();  
        System.out.println((o1 == o2) + " and " + (o1.equals(o2)));  
    }  
}
```

sol:

false false

4.

An __ is a class-like construct that contains only constants, abstract methods, default methods, and static methods. In many ways, an interface is similar to an abstract class, but an abstract class can contain __.

NOTE:

特性	Interface	Abstract Class
声明关键字	interface	abstract class
实例变量	No 不允许	Yes 可以有
构造方法	No 不允许	Yes 可以有
常量 (static final)	Yes 可以 (默认 public static final)	Yes 可以
抽象方法	Yes 可以 (默认 public abstract)	Yes 可以
default 方法 (Java 8+)	Yes 可以有方法体	N/A (普通方法即可)
static 方法 (Java 8+)	Yes 可以有方法体	Yes 可以有方法体
private 方法 (Java 9+)	Yes 可以 (辅助 default/static)	Yes 可以
继承	一个类可实现多个接口	一个类只能继承一个抽象类
方法默认访问修饰符	public	取决于声明
字段默认访问修饰符	public static final	取决于声明

实例初始化块	No	Yes
用途	定义”能做什么” (契约)	定义”是什么” (共性)

sol:

interface instance variables or constructors;

5.

The output from the following code is

```
public class Main {
    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<String>();
        list.add("New York");
        ArrayList<String> list1 = (ArrayList<String>) (list.clone());
        list.add("Atlanta");
        list1.add("Dallas");
        System.out.println(list1);
    }
}
```

note:

clone() 做的是:

1. 创建一个新的 ArrayList 对象 (新地址)
2. 把旧列表里的每个元素引用复制到新列表里

clone 前: list → [0x01(" New York")] // ArrayList 对象在 0xA list1 = null clone 后: list → [0x01(" New York")] // ArrayList 对象在 0xA list1 → [0x01(" New York")] // ArrayList 对象在 0xB (全新!) 所以

- list.add(" Atlanta") → 改了 list 这个 ArrayList 对象的结构, list1 是另一个 ArrayList, 不受影响。
- 共享的只是里面的" New York" 字符串引用, 但因为 String 不可变, 修改不了。

浅拷贝什么时候会”互相影响”? 如果存的是可变对象 (比如 ArrayList<Date>), 修改元素内部状态就会影响双方:

```
list1: [0x02(Date)] \
```

list: [0x02(Date), 0x03, ...] / 共享同一个 Date list.get(0).setTime(0); // 修改 Date 对象本身
list1.get(0).getTime(); // 也变了! 一句话: 浅拷贝 = 新列表新结构, 但元素引用共享。增删列表元素互不影响, 修改元素对象本身才会互相影响。

sol:

[New York, Dallas]

6.

The program below would encounter a compilation error because the Fruit class does not implement the java.lang.Comparable interface and the Fruit objects are not comparable. This is due to the problem in Line __;

```
public class Test {
    public static void main(String[] args) {
        Fruit[] fruits = { new Fruit(2), new Fruit(3), new Fruit(1) };
        Arrays.sort(fruits);
    }
}

class Fruit {
    private double weight;
    public Fruit(double weight) {
        this.weight = weight;
    }
}
```

sol:

4? the sort can not find the Fruit.compareTo() method to compare the Fruit objects;

7.

A __ class or method permits you to specify allowable types of objects that the class or method can work with. If you attempt to use a class or method with an incompatible object, the __ will detect the error.

sol:

generic compiler

8.

The method header is left blank in the following code. Fill in a generic method header __.

```
public class GenericMethodDemo {
    public static void main(String[] args ) {
        Integer[] integers = {1, 2, 3, 4, 5};
        String[] strings = {"London", "Paris", "New York", "Austin"};

        print(integers);
        print(strings);
    }
}
```

```

    }

    public static _____ {
        for (int i = 0; i < list.length; i++)
            System.out.print(list[i] + " ");
        System.out.println();
    }
}

```

sol:

```

public static <E> void print(E[] list) {
    for (int i = 0; i < list.length; i++)
        System.out.print(list[i] + " ");
    System.out.println();
}

```

9.

In Java collections, the _____ is efficient for retrieving elements and for adding and removing elements at the end of the list. On the other hand, _____ is better suited for adding and removing elements at any position within the list, although locating specific elements for operations is not as efficient.

note:

ArrayList 随机访问快, 尾部增删快
 LinkedList 已经定位到节点后, 中间插入/删除快; 但查找慢
 Stack LIFO 栈操作, 不是通用 List 的标准答案

sol:

ArrayList (not Stack) LinkedList

10.

Suppose that list1 is a list that contains the strings red, yellow, and green, and that list2 is another list that contains the strings red, yellow, and blue. After executing list1.remove(list2), list1 contains _____.

NOTE: require verify

sol:

red, yellow, green because list1.remove(list2) removes list2 object, not elements in list2 from list1. if we want to remove the common elements in list1 and list2, we should use list1.removeAll(list2);

11.

_____ store objects that are processed in a last-in, first-out fashion. _____ store objects that are processed in a first-in, first-out fashion.

sol:

Stack Queue

12.

The output from the following code is

```
public class Test {
    public static void main(String[] args) {
        PriorityQueue<Integer> queue = new PriorityQueue( Arrays.asList(20, 40, 60,
        ↪ 10, 50, 30));
        while (!queue.isEmpty()) {
            System.out.print(queue.poll() + " ");
        }
    }
}
```

sol:

10, 20, 30, 40, 50, 60

13. Show the output of the following code

```
Set<String> set = new LinkedHashSet<>();
set.add("ABC");
set.add("FGH");
System.out.println(set);
```

sol:

the LinkedHashSet maintains the insertion order [" ABC" , " FGH"]

14.

Complete the following code to create an empty hash set of integers: Set<Integer> set = new ____;

sol:

HashSet<>(); Set<Integer> is a declaration of the set variable, the Set is superclass of HashSet.

15.

In a library management system where you need to frequently check the availability and information of books with unique identifiers (e.g., the book ID), the most suitable data structure to store the books in the library would be a ____;

sol:

HashMap searching is O(1)

16.

The ____ is a theoretical approach for analyzing the performance of an algorithm. It estimates how fast an algorithm's execution time increases as the input size increases, which enables you to compare two algorithms by examining their growth rates.

sol:

Big O notation

17.

An algorithm with the __ time complexity is called a linear time algorithm and an algorithm with the __ time complexity is called a logarithmic algorithm.

sol:

$O(n)$ $O(\log n)$

18.

Use the Big O notation to estimate the time complexity of the following method:

```
public static void mD(int[] m) {  
    for (int i = 0; i < m.length; i++) {  
        System.out.print(m[i] + " ");  
    }  
}
```

sol:

$O(n)$

19.

The worst-case time complexity for selection sort, insertion sort, bubble sort, and quick sort algorithms is ____

sol:

$O(n^2)$

20.

Suppose a list is [12, 9, 15, 4, 20, 11]. After the first pass of bubble sort, the list becomes

sol:

[9, 12, 4, 15, 11, 20]

21. Indicate the number of possible values that can be put in the ‘?’ place to maintain the max heap

13

/ \

“?” 10

/ \

8 9

sol:

13-9

22.

In a graph, if two vertices are connected by two or more edges, these edges are called __. A __ is the one in which every two pairs of distinct vertices is connected by an edge.

sol:

parallel edge complete graph

23.

There are two popular ways to traverse a graph, namely and

sol:

BFS DFS

24.

In a network of cities connected by roads of varying lengths, if you want to connect all the cities in the most cost-effective way possible, you would use the __ algorithm. However, if you want to find the shortest route from one specific city to another, you would use the __ algorithm.

sol:

Prim's or Kruskal's algorithm (Minimum Spanning Tree) Dijkstra's algorithm

25.

The time complexity for inserting and deleting an element into a binary search tree __ are __ and respectively, assuming that the tree is balanced.

NOTE: to check the BinraySearchTree cpp implementation in the CPT204/BinarySearchTree

sol:

$O(\log n)$, $O(\log n)$

26.

In a binary tree, a __ traversal visits the root node first, followed by the left subtree, and then the right subtree. Conversely, a __ traversal visits the left subtree first, followed by the right subtree, and finally the root node..

sol:

preorder postorder

27.

In AVL tree, A node is said to be __ if its balance factor is -1, and it is said to be __ if its balance factor is +1.

note:

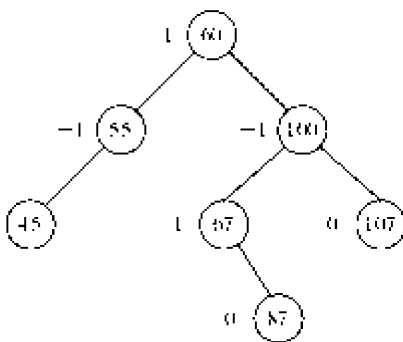
- balance factor = height(left subtree) - height(right subtree)
- BF = -1:
 - right subtree is higher than left subtree by 1
 - so the node is right-heavy (右重)
- BF = +1:
 - left subtree is higher than right subtree by 1
 - so the node is left-heavy (左重)

sol:

right-heavy left-heavy

28.

Given the original AVL tree below, after adding element 42, the rotation would be performed to rebalance the tree.

**sol:**

Right Rotation

29.

__ and __ methods are usually taken to deal with hashing collision where two different keys are mapped to the same index in a hash table.

sol:

Separate chaining Open addressing

30.

In hash tables, __ probing resolves collisions by sequentially checking the next available slots, while __ probing resolves collisions by checking slots at increasing squared distances from the original hash.

note:

- Linear probing (线性探测)
 - It resolves collisions by checking the next slots sequentially.
 - Probe sequence: $h(\text{key}), h(\text{key})+1, h(\text{key})+2, h(\text{key})+3, \dots$
 - Formula: $\text{index}_i = (h(\text{key}) + i) \bmod \text{table.length}$
- Quadratic probing (二次探测 / 平方探测)
 - It resolves collisions by checking slots at squared distances from the original hash position.
 - Probe sequence: $h(\text{key}), h(\text{key})+1^2, h(\text{key})+2^2, h(\text{key})+3^2, \dots$
 - Formula: $\text{index}_i = (h(\text{key}) + i^2) \bmod \text{table.length}$
- Probing (探测)
 - When a collision occurs, probing means searching for another available slot in the hash table.

sol:

Linear Probing, Quadratic Probing

Part B. Coding Questions

Question 1:

A real estate analyst is comparing the prices of recently sold luxury homes in a prestigious neighborhood. The prices of these homes are as follows: \$1,250,000.1, \$1,750,000.3, \$2,100,000.6, \$1,900,000.8, and \$2,300,000.2. Complete the following method that find the maximum element in an ArrayList of generic elements, return null if the ArrayList is empty, and return the maximum value of the ArrayList that was passed to the function if it is not null. `public static <E extends Comparable> E max(ArrayList list)`

```
import java.util.ArrayList;
import java.util.Arrays;

public class Main {
    public static <E extends Comparable<E>> E max(ArrayList<E> list) {
        // Empty set
        //
        // Highest price case
        return highestPrice;
    }

    public static void main(String[] args) {
        // printing Test case with an empty list
        ArrayList<Double> prices2 = new ArrayList<>();
        System.out.println(max(prices2)); // -> null

        // Printing Test case with highest prices
        ArrayList<Double> prices1 = new ArrayList<>({
            Arrays.asList(1250000.1, 1750000.3, 2100000.6, 1900000.8, 2300000.2));
        System.out.println(max(prices1));
    }
}
```

sol:

```
import java.util.ArrayList;
import java.util.Arrays;

public class Main {
    public static <E extends Comparable<E>> E max(ArrayList<E> list) {
        // Empty set
        if (list == null || list.size() == 0) {
            return null;
        }

        // Highest price case
        // E highestPrice = list.get(0);
        // for (int i = 1; i < list.size(); i++) {
        //     if (list.get(i).compareTo(highestPrice) > 0) {
        //         highestPrice = list.get(i);
        //     }
        // }
        // or use collection max method
        var highestPrice = java.util.Collections.max(list);
        // or using stream
        // var highestPrice = list.stream().max(E::compareTo).orElse(null);

        return highestPrice;
    }

    public static void main(String[] args) {
        // printing Test case with an empty list
        ArrayList<Double> prices2 = new ArrayList<>();
        System.out.println(max(prices2)); // -> null

        // Printing Test case with highest prices
        ArrayList<Double> prices1 = new ArrayList<>()
            Arrays.asList(1250000.1, 1750000.3, 2100000.6, 1900000.8, 2300000.2));
        System.out.println(max(prices1));
    }
}
```

Question 2:

You are tasked with enhancing a flight boarding system for an airline. The airline categorizes passengers into three classes: “First Class,” “Business Class,” and “Economy Class.” The following three priority queues represent the passengers waiting to board: First Class Queue: { “Alice”, “Bob”, “Charlie”, “Diana”, “Ethan”, “Fiona” } Business Class Queue: { “Charlie”, “Diana”, “Grace”, “Ian” } Economy Class Queue: { “Alice”, “Grace”, “Ethan”, “Hannah”, “Ian” } Your job is to find the exclusive First Class passengers who are only in “First Class” and not in “Business Class” or “Economy Class”, by completing the 3 tasks indicated in the following white board.

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashSet;
import java.util.PriorityQueue;
import java.util.Set;
```

```

public class Main {

    public static void main(String[] args) {
        // Task 1: Create PriorityQueues for each class of Passengers

        // Task 3: Call findExclusiveFirstClassPassengers method and print out hte
        // exclusive First class Passengers
    }

    public static Set<String> findExclusiveFirstClassPassengers(
        PriorityQueue<String> firstClassQueue,
        PriorityQueue<String> businessClassQueue,
        PriorityQueue<String> economyClassQueue) {
        // Task2: Implement this method to find exclusive First Class Passengers;
        // convert the queues into set for easier comparison
        return exclusiveFirstClass;
    }
}

```

sol:

```

import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashSet;
import java.util.PriorityQueue;
import java.util.Set;

public class Main {

    public static void main(String[] args) {
        // Task 1: Create PriorityQueues for each class of Passengers
        PriorityQueue<String> firstClassQueue = new PriorityQueue<>(
            Arrays.asList("Alice", "Bob", "Charlie", "Diana", "Ethan", "Fiona"));
        PriorityQueue<String> businessClassQueue = new PriorityQueue<>(
            Arrays.asList("Charlie", "Diana", "Grace", "Ian"));
        PriorityQueue<String> economyClassQueue = new PriorityQueue<>(
            Arrays.asList("Alice", "Grace", "Ethan", "Hannah", "Ian"));

        // Task 3: Call findExclusiveFirstClassPassengers method and print out hte
        // exclusive First class Passengers
        Set<String> exclusiveFirstClassPassengers =
            ↪ findExclusiveFirstClassPassengers(firstClassQueue,
                businessClassQueue, economyClassQueue);
        System.out.println("Exclusive First Class Passengers num: " +
            ↪ exclusiveFirstClassPassengers.size());
        ↪ exclusiveFirstClassPassengers.stream().forEach(i -> System.out.printf("%s ",
            i));
    }

    public static Set<String> findExclusiveFirstClassPassengers(
        PriorityQueue<String> firstClassQueue,
        PriorityQueue<String> businessClassQueue,
        PriorityQueue<String> economyClassQueue) {
        // Task2: Implement this method to find exclusive First Class Passengers;
        // convert the queues into set for easier comparison
    }
}

```

```
Set<String> exclusiveFirstClass = new HashSet<>(firstClassQueue);
Set<String> businessClassSet = new HashSet<>(businessClassQueue);
Set<String> economyClassSet = new HashSet<>(economyClassQueue);
var retainBusiness = exclusiveFirstClass.removeAll(businessClassSet);
var retainEconomy = exclusiveFirstClass.removeAll(economyClassSet);

return exclusiveFirstClass;
}
```